OpenAI Codex CLI Cheat Sheet

Lightweight coding agent (built in Rust) that reads, edits, and runs code from your terminal.

Core Commands

`codex`	Launch interactive TUI
`codex "prompt"`	Start session with initial prompt
`codex exec "prompt"`	Non-interactive, output to stdout
`codex exec -json "..."`	Emit newline-delimited JSON events
`codex resume -last`	Reopen most recent session
`codex resume <ID>`	Resume a specific session
`codex fork <ID>`	Fork session into a new thread
`codex apply <TASK>`	Apply cloud task diffs to local repo

Approval Modes & Sandbox

`-sandbox read-only`	Read but not write or execute
`-sandbox workspace-write`	Edits within working dir (default)
`-sandbox danger-full-access`	Unrestricted filesystem access
`-ask-for-approval untrusted`	Approve every command
`-ask-for-approval on-request`	Approve only when Codex asks
`-ask-for-approval never`	No approval prompts
`-full-auto`	Low-friction local work shortcut
`-yolo`	Bypass all safety checks

Change mid-session with `/permissions`.

Customization: AGENTS.md

Persistent instructions loaded once per session, concatenated root-to-cwd.

Scope	Path
Global	`~/.codex/AGENTS.md`
Global override	`~/.codex/AGENTS.override.md`
Project	`<repo-root>/AGENTS.md`
Subdirectory	`<subdir>/AGENTS.md`

- Override files at any level take precedence over base files
- Max combined size: 32 KiB (`project_doc_max_bytes`)
- Scaffold: `/init` Verify: `codex status`

Customization: config.toml

Located at `~/.codex/config.toml` (user) or `.codex/config.toml` (project). CLI `-c key=value` overrides both.

```
model = "gpt-5.4"
model_reasoning_effort = "high"
# minimal | low | medium | high | xhigh
personality = "pragmatic"
# none | friendly | pragmatic
sandbox_mode = "workspace-write"
web_search = "live"
# disabled | cached | live
file_opener = "vscode"
# vscode | cursor | windsurf | none

[history]
persistence = "save-all"

[agents]
max_threads = 6
max_depth = 3
```

Profiles

```
profile = "default"

[profiles.careful]
sandbox_mode = "read-only"
model_reasoning_effort = "xhigh"
```

Load with `codex -p careful` or `-profile careful`.

Skills

Reusable instruction bundles. Detected automatically or invoked explicitly with `$skill-name`.

Action	How
Invoke	`$skill-name` in prompt
Create	`$skill-creator` or manual `SKILL.md`
Install	`$skill-installer linear`
Disable	`enabled = false` in config.toml

Skill Locations (searched in order)

Scope	Path
Repo (local)	`./.agents/skills/`
Repo (root)	`$REPO_ROOT/.agents/skills/`
User	`~/.agents/skills/`
Admin	`/etc/codex/skills/`
System	Bundled with Codex

SKILL.md Structure

```
---
name: my-skill
description: When to trigger this skill
---
Step-by-step instructions for Codex
```

Optional dirs: `scripts/`, `references/`, `assets/`, `agents/openai.yaml`.

Key Slash Commands

`/review`	Review working tree changes
`/plan`	Switch to plan mode
`/model`	Change model and reasoning effort
`/fast`	Toggle fast mode (GPT-5.4)
`/diff`	Git diff including untracked files
`/mention`	Attach a file to conversation
`/compact`	Summarize conversation to save tokens
`/mcp`	List configured MCP tools
`/apps`	Browse and insert connectors
`/fork`	Fork conversation into new thread
`/clear`	Clear terminal, fresh chat
`/status`	Session config and token usage
`/init`	Generate AGENTS.md scaffold

MCP Integration

Add external tools via Model Context Protocol servers in config.toml.

```
[mcp_servers.my-server]
command = "npx my-mcp-server"
# url = "https://..."  # HTTP server
enabled_tools = ["tool_a", "tool_b"]
startup_timeout_sec = 15
required = true
```

Manage with `codex mcp` subcommand.

Automation with exec

```
# Write result to file
codex exec -o result.md "summarize changes"

# Validate JSON output against schema
codex exec --output-schema schema.json \
  "generate config"

# Ephemeral run (no session saved)
codex exec --ephemeral "lint and fix src/"

# Skip git repo check
codex exec --skip-git-repo-check \
  "analyze logs in /tmp/data"

# Cloud execution with best-of-N
codex cloud exec --env ENV_ID \
  --attempts 4 "refactor auth module"
```

Tips & Shortcuts

`@` in composer	Fuzzy file search
`!command`	Run shell command inline
`Enter` while running	Inject instructions mid-turn
`Ctrl+G`	Open `$EDITOR` for long prompts
`Esc Esc`	Edit previous message / fork
`-i img.png`	Attach image for visual context
`-add-dir ../other`	Add extra dirs to workspace
`codex completion zsh`	Install shell tab completions